

Project Overview

The **goal** of my project was to find the quickest way to find a transitive win from one fighter to another using UFC fights since 2010. For example, if A beats B and B beats C then $A \rightarrow B \rightarrow C$

The **Dataset** is uploaded to github and is the link is here:

<https://www.kaggle.com/datasets/mdabbert/ultimate-ufc-dataset/data>

The source is [UFCstats.com](https://ufcstats.com) and has been scraped from there by a github user who uploaded it to kaggle. I ended up using there github data

(https://github.com/shortlikeafox/ultimate_ufc_dataset), because it had data from February of this year versus December of 2024. (More updated)

Data Processing

I loaded it into rust by downloading the CSV and uploading it to excel, where I removed a few columns I did not need such as streaks, gambling odds for finishes and more. I then downloaded the cleaned file as a CSV and moved it to my rust file. I used the dataframe methods we used in class to read the data and classify each entry as a type of data.

Code Structure:

Modules

My modules are readdata, winpath, and main. Readdata creates the function that reads a CSV, turns it into a dataframe, and also defines what a dataframe is. Since this is purely helping create structure and is not specific to my data, it is good to separate. Winpath establishes graph methods and construction functions and also makes the function that reads a graph and its directed edges and displays its transitive win path. Since winpath could be used to read really any directed graph with relevant identification, I separated it since someone may want just the function and not care about the UFC element of the data. The main module use the other two modules and processes the data into the data frame function and then creates a directed graph using the graph methods and applies the function.

Key Functions and Types:

In Read Data:

Dataframe is a struct with a vector of strings that are the labels and a vector of columnval filled vectors, where column val is an enum that is either a string, float, int, or bool. This constructs a struct with labels and data.

Read CSV takes an empty dataframe, a path to a CSV, and a vector of the types and returns a dataframe where the first row is string labels and every other row is additional data of one of four types (float, int, bool, or string) and is read by separating each entry by comma and then by line.

In Win Path:

I implement a Graph struct that takes n number of vertices and a vector of each edge, I add a function that creates an instance of a graph. There is also a function that pushes the second element in each tuple in the edge list to the first element's outedges list. I also created a sort method. I then used a create directed function which takes the created outedges list and sorts each outedge list for each vertex and returns the sorted graph.

For the actual transitive win path function, it takes an argument of a graph, a name to ID hashmap and an ID to name hashmap. This allows me to assign each name to a vertex and

then after finding the path backtrack each vertex to a name. It also takes a from name entry and a goal entry which are used as an origin and destination. First the function takes the name entries and checks to see if they exist in the hashmap of name_to_id, if they don't it returns a message saying the fighter was not found. It then initializes an empty vector to represent the parent nodes, a boolean false-filled vector that represents whether a vector has been visited, and a queue to add neighbors to the queue. While there are items in the queue, check if the current item is the goal fighter, if not check if there are unvisited neighbors and then add each unvisited neighbor to the back of the queue. If there is nothing left in the queue and the goal node has not been visited it prints a statement that no path was found. If it finds a path it adds each node on the path to a vector and then adds the start vector. It then reverses the order to show the actual path and maps the ID numbers back to names. Then it prints that statement.

In main, it reads the data in the file and uses the csv function to turn it into a dataframe. It then reads the winner and loser columns and makes them case insensitive. For each winner and loser entry it gives the fighter a unique ID if they weren't already entered and turns that into the fighter to ID hashmap. It also creates a counter to count the total number of unique fighters, it then pushes each new ID to an edge list. It then creates a graph using the IDs and counter. It then creates a hashmap that will backtrack the ID to the name after the path is found. It then uses the name typing to get fighter names. It then calls the function.

Tests:

For tests it first tests a basic graph and ensures that the outedges methods work and that they return the expected result, if the outedges did not properly track in the directed graph it would improperly find win paths. Then it checks the name to id and id to name hashmaps idea to ensure it will return the expected results, if these did not work it could lead to mislabeling of fighters. Lastly, it takes the two hashmaps and creates a very simple two node graph to ensure that the function will run, this is vital to make sure the function returns the expected result.

Output:

running 4 tests

test tests::test_id_to_name_mapping ... ok

test tests::test_basic_graph ... ok

test tests::test_name_to_id_mapping ... ok

test tests::test_path_does_not_panic ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Results:

Example:

```
Enter winner's name: Jon Jones
Enter target loser's name: Henry Cejudo
Transitive win path: ["Jon Jones", "Dominick Reyes", "Chris Weidman", "Bruno Silva", "Tyson Nam", "Zarrukh Adashev", "Ryan Benoit", "Sergio Pettis", "Joseph Benavidez", "Henry Cejudo"]
```

```
Enter winner's name: Jon Jones
Enter target loser's name: Ilia Topuria
No transitive win path from 'Jon Jones' to 'Ilia Topuria'.
```

I enjoyed playing around with it trying to find ways very small fighters beat very big fighters, or bad fighters beat good fighters. I think it highlights the variance in results in the UFC.

Usage Instructions:

Ensure the UFC.Master dataset (or other dataset with winner and loser columns) is in folder. If the column numbers do not match the current dataset change the types for each column on line 17 in [main.rs](#) and the loop that reads each row on lines 26 and 30 in [main.rs](#)

Run code and then type out desired winner (starts) name and press enter and then desired loser (ends name) press enter. The entry is not case sensitive, but you must type the fighter's name correct.

On my computer it never ran longer than .5 seconds using cargo run, but varied depending on the length of the path.

Citations:

I referenced some lectures for my DS210 course at Boston University with Professor Gardos, in some cases I used pre-built structs, enums and functions from the course.